AXI Tutorial

AXI

Advanced eXtensible Interface (AXI), is an interface specification/protocol for communicating between a master and a slave. In this exercise, a GPIO device will be used as the slave, and a master will be developed. Most of the Xilinx IP (Intellectual Property) available in Vivado interfaces over AXI, so knowing how to use it is crucial for larger designs where not all of the components are written from scratch.

There are three variants of AXI: Full, Lite, and Stream. AXI Full is the most complicated, supporting operations such as burst reads and block memory access. Lite will be used in this exercise, and is a simple way of interfacing with memory mapped components. Finally, Stream is meant for fast data transfers without regard for an address, such as to and from a FIFO.

The AXILite protocol consists of five different channels, each with their own set of control signals, as shown in Figure 1.

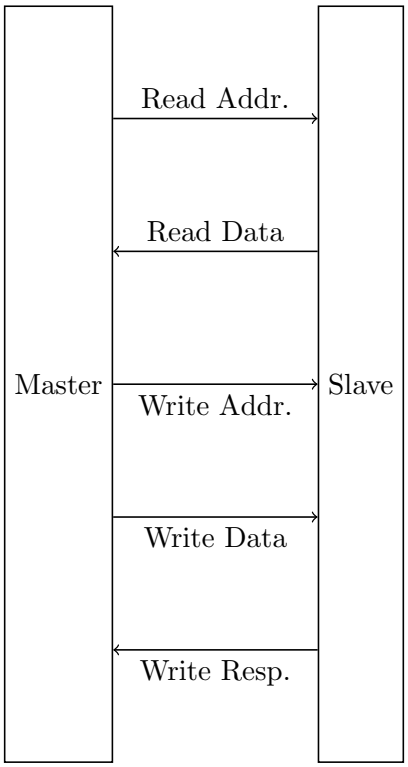


Figure 1: AXI Transaction Channels

The arrow indicates the direction of data flow, but some of the control signals go in the opposite direction. Transactions are always initiated by the master, so to perform a read, the master would first send a `read address`, then the slave would send the data back over the `read data` channel.

Writes are performed by the master sending the `write address` and `write data`, either concurrently or sequentially, and then the slave returning a `write response` to inform the master whether the transaction was successful.

Each channel has two primary control signals, `ready` and `valid`. The `valid` signal is always asserted

by the component sending the data, while the `ready` signal is controlled by the receiver. Initially, the ready signals can either be high or low, but high is preferred. Both signals need to be high in order for the transfer to take place, making the transfer one cycle more efficient if `ready` is high. Once that has happened, the control signals can, but do not have to be deasserted.

In other words, if data can be sent, the sender should put that data on the bus and assert `valid`. A sample transaction is shown in Figure 2.

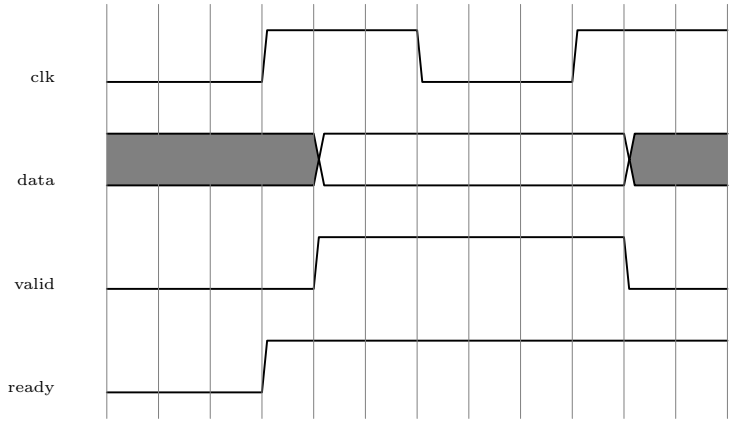


Figure 2: Sample AXI Transaction

The data is transferred on the second rising edge of the clock. Here, `ready` remains high to indicate that another transaction can be performed. However, because `valid` was pulled low, no transfer will take place on the next clock cycle. To prevent deadlocks, the `valid` signal should not be dependent on `ready`. For further details, the AXI specification is posted on MyCourses.

Setting up

1. Download the AXITut.tar file from MyCourses. Unarchive it as follows:

   ```
   tar -xvf AXITut.tar
   ```

2. Start the Xilinx Vivado software and open the AXITut project in the unarchived folder. Note that **axi_top.vhd** will have a syntax error, this is expected.

Generating IP

3. Click on **IP Catalog** under **Project Manager** and search for "AXI GPIO". This IP allows connecting the switches and LEDs via an AXILite interface.

4. Double click on the result to open a new window which allows for customizing the IP core.

5. Select the **IP Configuration** tab and check the **Enable Dual Channel** box. This will add a second GPIO interface to the diagram.

6. Select the first GPIO to be **All Inputs**, and change the width to be **16**. This GPIO will be connected to the switches

7. GPIO 2 needs to be **All Outputs**, and also have a width of **16**. It will interface with the LEDs.

8. Click the plus next the **GPIO** and **GPIO 2** to see the signals in the interface. The Customize IP window should now look like Figure 3.

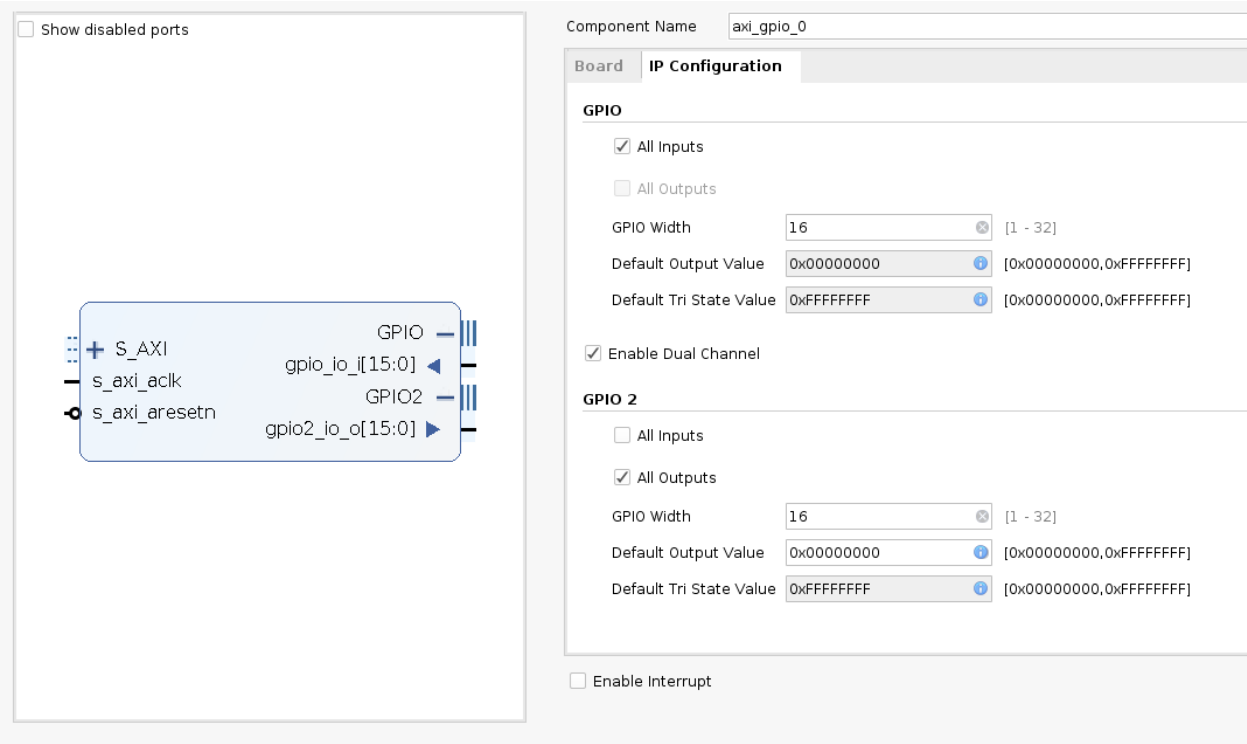


Figure 3: GPIO Final Customization

9. Press **OK**, make sure **Out of context per IP** (OOC) is selected, the press **Generate**. OOC will speed up synthesis in the future if one IP changes but not others.

Using IP

10. Locate the **Sources** pane, and click on **IP Sources** at the bottom of it. Expand **axi_gpio_0** and then Instantiation Template. Open **axi_gpio_0.vho**.

11. Open **axi_top.vhd** and find the comment that says `AXI GPIO Component`. Copy the component declaration from **axi_gpio_0.vho**. Find the instantiation template and copy it to the comment which reads `AXI GPIO Instantiation`. **Important:** Change the name of the instance to "`axi_gpio_inst_0`".

12. In the instantiation, map the `s_axi_aclk` to `CLK_100MHZ`. `gpio_io_i` should be connected to `SWITCHES`, and `gpio2_io_o` to the `LEDs`.

13. Locate the signal assignments of `rd_addr` and `wr_addr`. Use the **Register Space** section of the AXI GPIO Product Guide (PG) to determine what they should be. The PG is available at `https://www.xilinx.com/support/documentation/ip_documentation/axi_gpio/v2_0/pg144-axi-gpio.pdf`. Remember that the switches are `GPIO` and the LEDs are `GPIO 2`.

Simulation

14. Run a behavioral simulation. Be sure that there are no messages indicated that `Read or write failed`.

15. Open the waveform and add the `s_axi` interface from the **Protocol Instances** pane. The signals are recognized as a protocol because of the "axi_top.protoinst" file included as a simulation source.

16. Restart and run the simulation and expand the AXI interface (it may say "Loading..." for a moment before it is ready to be viewed). Expand the "Channels" drop down. Zoom in on one write transaction and expand the Read and Write groups to examine the actual signals.

Hardware & ILA Debug

17. Open the Nexys4.xdc file and scroll down past the pin mappings. There are several lines describing debug cores and ILA probes. These lines were generated from following the ILA generation process (which was done before compressing the project).

18. Close the Nexys4.xdc file and **Generate Bitstream**.

19. Program the board, but instead of using the default "Debug probes file", click the ellipses symbol and navigate to the top level of the project and select "axi_top_mod.ltx". This file was modified to recognize the AXI signals as a protocol, similar to the simulation .protoinst file.

20. Ensure that flipping a switch and then pressing the lower button causes the corresponding LED to change. Test the reset capability as well by pressing the uppermost button.

21. In the ILA window, add a trigger for when AWVALID is high.

22. Run the trigger for the ILA core and observe the AXI transactions. The signals will be grouped together under the "slot_0 : M_AXI : Interface" group like they were in the simulation. Change the switch positions and press the lower button and notice how the values of `s_axi_rdata` and `s_axi_wdata` change.

State Machine Diagrams

23. Draw state diagrams for the state machines in the provided files.

AXI handshake

24. Draw and annotate waveforms for a sample AXILite transaction (similar to the handshake example given in the State Machine tutorial).

Tutorial 4: AXI Tutorial

Student's Name:_____

Demo	Point Value	Points Earned	Date
Part 1: Complete behavioral simulation of AXILite system (explain purpose of five AXI Lite channels)	25		
Part 2: ILA capture of read/write on AXILite system	25		
Part 3: Drawings of state machine diagrams in the provided design files	25		
Part 4: Annotated waveforms for a sample AXILite transaction - explain each step in the handshake	25		

Students will need to demonstrate their understanding of all parts of this Tutorial by answering questions from Instructor/TAs.

Submit this grading sheet on myCourses under Assignemnts → Tut4.